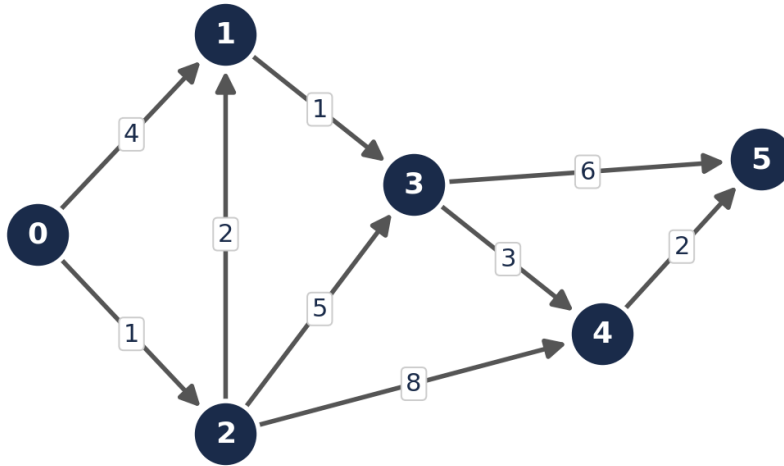


25 Bài Tập Thuật Toán Dijkstra

Bài 1. Biểu diễn đồ thị có trọng số

Cho **đồ thị có hướng** bên dưới (trọng số dương). Hãy xây dựng **danh sách kề** (adjacency list). Đây là bước chuẩn bị cho mọi bài Dijkstra phía sau — các bài 1–8 đều dùng chung đồ thị này.



Đồ thị $G1$ — dùng chung cho các bài 1–8 (đỉnh 0–5, trọng số trên cạnh).

Ví dụ: $\text{adj}[0]=[(1,4), (2,1)]$; $\text{adj}[2]=[(1,2), (3,5), (4,8)]$

Bài 2. Tính tay đường đi ngắn nhất

Với đồ thị $G1$ ở Bài 1, **chạy tay** thuật toán Dijkstra từ nguồn $s = 0$. Lập bảng ghi lại $\text{dist}[]$ sau mỗi vòng chọn đỉnh và thứ tự các đỉnh được "chốt".

Ví dụ: thứ tự chốt: 0, 2, 1, 3, 4, 5

Bài 3. Dijkstra cơ bản: nguồn tới mọi đỉnh

Cài đặt Dijkstra (bản $O(V^2)$) tìm khoảng cách ngắn nhất từ nguồn $s = 0$ tới **tất cả** các đỉnh của $G1$.

Ví dụ: $s = 0 \rightarrow \text{dist} = [0, 3, 1, 4, 7, 9]$

Ràng buộc: trọng số ≥ 0

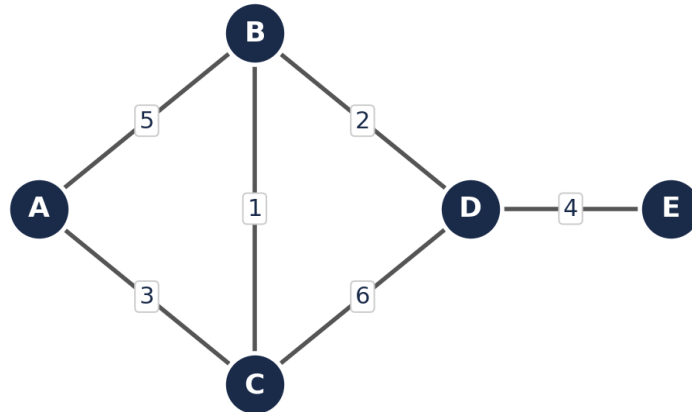
Bài 4. In khoảng cách tới mọi đỉnh

Sau khi chạy Dijkstra, in $\text{dist}[i]$ cho mọi đỉnh; in **-1** (hoặc ∞) nếu đỉnh không tới được từ nguồn.

Ví dụ: $\text{dist}[5] = 9$; đỉnh không nối $\rightarrow -1$

Bài 5. Đồ thị vô hướng có trọng số

Áp dụng Dijkstra cho **đồ thị vô hướng** các thành phố bên dưới. Tìm khoảng cách ngắn nhất từ thành phố A tới các thành phố còn lại.



Đồ thị vô hướng G2 — các thành phố A-E.

Ví dụ: từ A $\rightarrow$ A=0, C=3, B=4, D=6, E=10

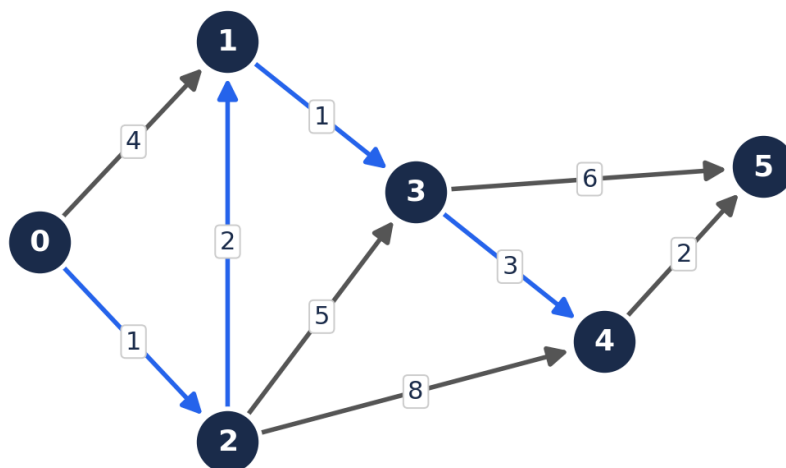
Bài 6. Đường đi ngắn nhất giữa hai đỉnh

Trên G1, tìm độ dài đường đi ngắn nhất từ s = 0 tới t = 4. Có thể dừng sớm khi lấy được t ra khỏi hàng đợi.

Ví dụ: s = 0, t = 4 $\rightarrow$ 7

Bài 7. Truy vết đường đi (reconstruct path)

Dùng mảng parent[] để truy ngược và in ra **dãy đỉnh** trên đường đi ngắn nhất từ 0 tới 4 của G1 (đường tô xanh bên dưới).



Đường đi ngắn nhất 0 $\rightarrow$ 2 $\rightarrow$ 1 $\rightarrow$ 3 $\rightarrow$ 4 được tô xanh.

Ví dụ: 0 $\rightarrow$ 2 $\rightarrow$ 1 $\rightarrow$ 3 $\rightarrow$ 4 (độ dài 7)

Bài 8. Số đỉnh trong bán kính D

Trên G1, đếm số đỉnh có khoảng cách ngắn nhất từ nguồn 0 thỏa $\leq D$.

Ví dụ: D = 3 $\rightarrow$ 3 đỉnh (0, 2, 1)

Bài 9. Dijkstra với hàng đợi ưu tiên

Cài đặt Dijkstra dùng **priority queue / min-heap**, đạt độ phức tạp $O((V + E) \log V)$. So sánh tốc độ với bản $O(V^2)$ trên đồ thị thưa.

Ví dụ: $n=10^5, m=2 \cdot 10^5 \rightarrow$ cần bản heap
Ràng buộc: $1 \leq n, m \leq 2 \cdot 10^5$

Bài 10. Chọn cài đặt theo mật độ đồ thị

Cho biết khi nào nên dùng bản $O(V^2)$ (đồ thị dày) và khi nào dùng heap (đồ thị thưa). Chứng minh bằng phân tích độ phức tạp theo V và E .

Ví dụ: $E \approx V^2 \rightarrow O(V^2)$ tốt; $E \approx V \rightarrow$ heap tốt

Bài 11. Nhiều nguồn (multi-source)

Cho một tập đỉnh nguồn. Tìm với mỗi đỉnh khoảng cách tới **nguồn gần nhất**. Gợi ý: thêm một "siêu nguồn" nối tới mọi nguồn với trọng số 0.

Ví dụ: nguồn = {0, 3} $\rightarrow$ dist[i] = min tới 0 hoặc 3

Bài 12. Đi qua đỉnh bắt buộc

Tìm đường đi ngắn nhất từ s tới t **bắt buộc đi qua** đỉnh k . Gợi ý: $\text{dist}(s, k) + \text{dist}(k, t)$.

Ví dụ: $s=0, t=5$, qua $k=2$

Bài 13. Đếm số đường đi ngắn nhất

Đếm **số lượng đường đi ngắn nhất** khác nhau từ nguồn tới mỗi đỉnh (cùng độ dài tối thiểu).

Ví dụ: có 2 đường cùng dài 7 tới đỉnh $t \rightarrow 2$

Bài 14. Đường đi ngắn nhì

Tìm độ dài **đường đi ngắn nhì** (second shortest, có thể dùng lại cạnh) từ s tới t . Gợi ý: lưu hai khoảng cách tốt nhất cho mỗi đỉnh.

Ví dụ: ngắn nhất = 7, ngắn nhì = 9

Bài 15. Dijkstra trên lưới (grid)

Cho lưới chi phí bên dưới (chi phí khi bước vào mỗi ô). Đi 4 hướng, tìm **tổng chi phí nhỏ nhất** từ ô trên-trái (S) tới ô dưới-phải (đích).

1	3	1
1	5	1
4	2	1

Lưới chi phí 3×3 (ô tô xanh là điểm xuất phát và đích).

Ví dụ: lưới 3×3 bên dưới $\rightarrow$ tổng nhỏ nhất tới đích

Bài 16. Trọng số trên đỉnh

Chi phí nằm ở **đỉnh** thay vì cạnh. Biến đổi bài toán để áp dụng Dijkstra (ví dụ tách mỗi đỉnh thành cạnh vào-ra).

Ví dụ: chi phí đi qua đỉnh $v = c[v]$

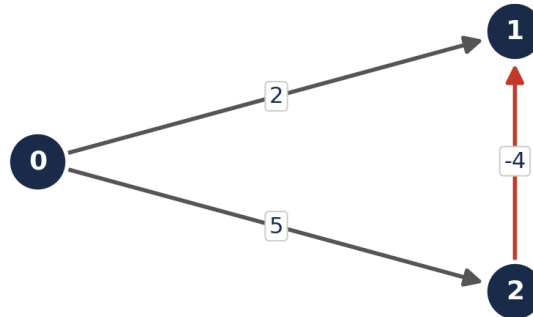
Bài 17. Đường đi bottleneck (minimax)

Tìm đường đi từ s tới t sao cho **cạnh lớn nhất trên đường là nhỏ nhất** (minimax / widest-path biến thể). Sửa phép "relax" của Dijkstra cho phù hợp.

Ví dụ: minimize max edge weight on path

Bài 18. Vì sao Dijkstra cần trọng số không âm

Xét đồ thị bên dưới có một **cạnh âm** (2→1 trọng số -4). Chạy Dijkstra từ 0 và chỉ ra nó cho kết quả **sai** ở đỉnh nào, giải thích bước "chốt đỉnh" bị phá vỡ ra sao. Nêu thuật toán thay thế.



Cạnh âm 2→1 (đỏ) khiến Dijkstra chốt sai dist[1]. Cần dùng Bellman-Ford.

Ví dụ: Dijkstra: dist[1]=2 (sai) ; thực tế 0→2→1 = 1

Bài 19. Đường đi xác suất lớn nhất

Mỗi cạnh có xác suất "thành công" trong [0,1]. Tìm đường đi từ s tới t có **tích xác suất lớn nhất**. Gợi ý: dùng -log để đưa về bài cộng trọng số, hoặc đổi phép relax sang nhân/lấy max.

Ví dụ: maximize product of probabilities

Bài 20. K đường đi ngắn nhất

Tìm độ dài của **K đường đi ngắn nhất** (cho phép lặp đỉnh) từ s tới t. Gợi ý: Dijkstra mở rộng cho phép một đỉnh được lấy ra tối đa K lần.

Ví dụ: K=3 → [7, 9, 10]

Bài 21. Dijkstra trên trạng thái mở rộng

Trạng thái = (đỉnh, thông tin phụ) — ví dụ lượng nhiên liệu còn lại, hoặc số vé miễn phí. Tìm chi phí nhỏ nhất tới đích trên đồ thị trạng thái mở rộng.

Ví dụ: state = (đỉnh, nhiên_liệu)

Bài 22. Giới hạn số cạnh trung chuyển

Tìm đường đi rẻ nhất từ s tới t dùng **tối đa k cạnh** (k điểm dừng). Gợi ý: Dijkstra/BFS phân lớp theo số cạnh, trạng thái (đỉnh, số_cạnh_đã_dùng).

Ví dụ: k=1 điểm dừng → đường rẻ nhất ≤ 2 cạnh

Bài 23. Nhiều truy vấn đường đi ngắn nhất

Cho nhiều truy vấn (s, t). Thảo luận chiến lược: chạy Dijkstra mỗi truy vấn vs tiền xử lý (ví dụ Dijkstra từ mỗi nguồn cần thiết) và đánh đổi thời gian/bộ nhớ.

Ví dụ: Q truy vấn → chọn chiến lược phù hợp

Bài 24. Dijkstra vs Bellman-Ford vs A*

So sánh ba thuật toán: điều kiện áp dụng (cạnh âm?), độ phức tạp, và vai trò của **hàm heuristic** trong A*. Cài A* với heuristic admissible trên lưới và so số đỉnh đã duyệt.

Ví dụ: A* duyệt ít đỉnh hơn Dijkstra nếu heuristic tốt

Bài 25. Chứng minh tính đúng đắn của Dijkstra

Phát biểu và chứng minh **bất biến tham lam**: khi một đỉnh được lấy ra khỏi hàng đợi, khoảng cách của nó đã là tối ưu. Chỉ rõ vì sao chứng minh này **cần** trọng số không âm.

Ví dụ: Bài chứng minh, không cần code